INT161 · Basic Backend Development

RESTful API

REST constraints, การตั้งชื่อ resource ด้วย URI และการแบ่ง code เป็น layer โดยไม่ใช้ framework

What is REST | Principles and Constraints | Uniform Interface | URI Naming

Layered System | Layers in Pure Node.js | อภิธานศัพท์

01

What is REST

นี่คือสิ่งที่ทั้งวิชาสร้าง: **REST API** ที่คุยกับ **front end** ด้วย **JSON** เครื่องมือเปลี่ยนได้ แต่โครงเหมือนเดิม

- REST ย่อจาก REpresentational State Transfer นิยามโดย Roy Fielding ผู้มีส่วนร่วมในการออกแบบ **HTTP**
- REST API กับ **RESTful** API ใช้ปนกัน อาจารย์อ่านว่า RESTful คือทำตามกฎครบ และชื่อสื่อถึงความ "สบาย" หลังยุค **SOAP** ที่หนัก
- API ย่อจาก Application Programming Interface: ส่วนของโปรแกรมที่โปรแกรมอื่นเรียกใช้ ของเราถูกเรียกโดย front end
- REST **server** ให้เข้าถึง **resource** สำหรับเราคือ table ใน database (**entity**) แต่จะเป็นไฟล์ หรือแม้แต่แอร์ที่คุมด้วย IoT ก็ได้
- สิ่งที่ front end ทำกับ resource คือ **CRUD**: create คือ INSERT, read คือ SELECT, update คือ UPDATE, delete คือ DELETE
- แต่ละ resource ตั้งชื่อด้วย **URI** รูปแบบข้อมูลที่ส่งระหว่างสองฝั่งเป็น text, XML หรือ JSON ก็ได้ แต่ทุกคนใช้ JSON

> **Boilerplate**
> เขียน CRUD ให้สิบ table จะได้ code เดียวกันสิบชุด เปลี่ยนแค่ table กับ field code ที่ซ้ำแบบนี้เรียกว่า boilerplate **framework** ช่วยแก้ปัญหานี้ คลาสนี้ยังไม่มี framework จึงให้รู้สึกถึงปัญหาก่อน

front end อยู่ที่ไหน มันถูก deploy บน server ของมันเอง **request** แรกไปที่นั่นและโปรเจกต์ front end ทั้งก้อนถูกโหลดมาที่ **browser** หลังจากนั้น browser จึงเรียก **back end** (browser จะตรวจการเรียกข้าม

origin นั่นคือ CORS สอนในวิชาอื่น ตอนทดสอบด้วย **Postman** ไม่มี front end จึงไม่มี CORS)

02

Principles and Constraints

`Client-server`

สองโปรแกรมแยกกัน back end ให้ resource front end ขอใช้

`Stateless`

server ไม่จำ **client** แต่ละ request พกสิ่งที่ต้องใช้มาเอง

`Uniform interface`

ทุกคนเรียก API แบบเดียวกัน เป็นข้อที่เราเรียนละเอียด

`Layered system`

แบ่ง code เป็น layer ตามหน้าที่ อีกข้อที่เราเรียนละเอียด

`Cacheable`

response บอกว่านำกลับมาใช้ซ้ำได้ไหม วิชานี้ข้ามไป

`Code on demand`

ไม่บังคับ server ส่ง code ให้ client รันได้ ทำน้อย

- **Stateless** แบบละเอียด application state คือสิ่งที่แอปจำเกี่ยวกับผู้ใช้แต่ละคน เช่น ใคร login อยู่ มีอะไรในตะกร้า แบบหลายหน้าแบบเก่า server เก็บใน session แต่ใน REST back end ไม่เก็บ front end เป็นคนเก็บและส่งสิ่งที่ต้องใช้มาทุก request
- ทำให้เขียน front end ยากขึ้น แต่ back end ง่ายและขยายได้ง่าย
- **Cacheable** เทียบกับตู้เย็น: ตลาดคือ storage ตู้เย็นคือ memory เก็บของที่ทำบ่อยไว้ในตู้เย็น hit คือมีของอยู่ ถ้าเดาผิดก็เปลืองที่ จึงต้องเดาแม่น library ทำให้ได้

03

Uniform Interface

Uniform แปลว่าเครื่องแบบ วันจันทร์ชุดนักเรียน วันพฤหัสชุดลูกเสือ ไม่ต้องคิดว่าจะใส่อะไร ที่นี่ก็เช่นกัน นักพัฒนา front end กับ back end ไม่ต้องประชุมเพื่อตกลงวิธีเรียก API เพราะมาตรฐานกำหนดไว้แล้ว

1. ระบุ resource

ด้วย URI ที่เป็นคำนาม ไม่ใช่ action

2. ส่ง representation

รูปแบบมาตรฐาน คือ JSON และ Content-Type เป็น application/json

3. ข้อความอธิบายตัวเอง

HTTP method บอกว่าทำอะไร **header** อธิบายข้อมูล

4. HATEOAS

response แนบ link ของ action ถัดไป ไม่บังคับ และมีคนทำน้อย

URI ตั้งชื่อ resource method เลือก action ชื่อเดียว สี่ action ลองเลือกคู่ดู

Method: **GET** | POST | PUT | PATCH | DELETE

เป้าหมาย: **Collection** | รายการเดียว

```
GET /api/v1/products
```

เทียบเท่า SQL	JSON body	สำเร็จ
`SELECT * FROM products`	ไม่ต้อง	`200 OK`

ดึงสินค้าทั้งหมด ได้ JSON array ตารางใหญ่ควรแบ่งหน้า

- ก่อนมีมาตรฐาน front end เรียก /create-user ตอนนี้เรียกที่ resource แล้วเลือก action ด้วย method
- หลายทีมลบโดยทำเครื่องหมายว่าลบแล้วแทน (**soft delete**) เพื่อให้ตามย้อนข้อมูลได้
- request ที่สร้างหรืออัปเดตต้องมี Content-Type เป็น application/json

HATEOAS: data plus links

```
{
  "id": 1,
  "name": "Somchai",
  "links": [
    { "rel": "orders", "href": "/api/v1/customers/1/orders" },
    { "rel": "delete", "href": "/api/v1/customers/1" }
  ]
}
```

04

URI Naming

endpoint คือ URI รวมกับ method กฎเหล่านี้คือสาเหตุที่คะแนนหาย อาจารย์เตือนว่า /product ที่ไม่มี s อาจเสียคะแนนในข้อสอบ

เลือก URI เพื่อดูผล

- `/api/v1/products`
- `/api/v1/getProducts`
- `/api/v1/Products`
- `/api/v1/product`
- `/api/v1/order_items`
- `/api/v1/products/`

```
/api/v1/devices.xml
```

```
/api/v1/customers/1/orders
```

- ใช้คำนาม ห้ามใช้ verb table เป็นคำนาม resource จึงเป็นด้วย
- พหูพจน์ทุกที่ แม้รายการเดียว /products, /products/1
- ตัวพิมพ์เล็ก และคั่นคำด้วยขีดกลาง (**kebab-case**) underscore เป็นธรรมเนียมของ database
- ใช้ / เพื่อลงลึก: /customers/1/orders คือ order ของ customer 1 ห้ามมี / ท้ายสุด ห้ามนามสกุลไฟล์ และห้ามมีคำ CRUD ใน path
- เริ่ม path ด้วยชื่อแอปและ version เช่น /api/v1 ถ้าภายหลังเปลี่ยนสิ่งที่ endpoint ส่ง ให้ออก /api/v2 แล้วให้แต่ละ front end ย้ายเมื่อพร้อม ตัวเก่ายังใช้ได้
- **collection** ตอบเป็น JSON array รายการเดียวตอบเป็น JSON object

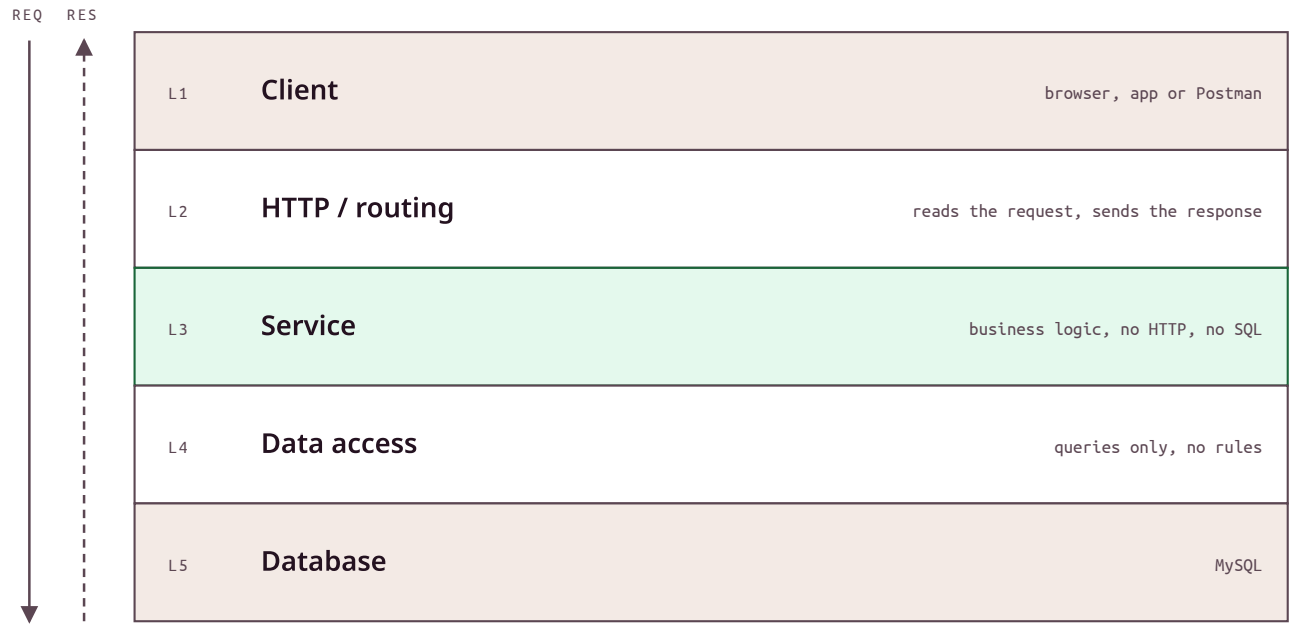
> ⚠ **ในโลกจริง**
> ไม่ใส่ version ก็ได้ถ้าคุมทั้งสองฝั่งและแก้พร้อมกัน แต่ front end ที่ใช้งานอยู่แล้วบังคับให้แก้ทันทีไม่ได้ การใส่ version จึงเป็นนิสัยที่ปลอดภัย

> 💡 **ใช้ AI**
> อาจารย์ให้ AI บน browser ออกแบบ endpoint ของ customer, product และ order ซึ่งตามกฎเดียวกัน: คำนาม ตัวพิมพ์เล็ก /api/v1 หนึ่ง method ต่อหนึ่ง action และ **status code** 200, 201, 400, 401 ใช้ AI อธิบายคำที่ไม่รู้ ไม่ใช่เขียน code ที่เราต้องเรียน และต้องรู้พอจะจับผิดได้

05

Layered System

layered system แบ่งโปรแกรมตามหน้าที่ นี่คือ separation of concerns: คนเขียน layer หนึ่งไม่ต้องสนใจ layer อื่น



HTTP / routing layer

คุยกับ front end อ่าน method และ **URL** แปลง JSON **body** เป็น JavaScript object และส่ง response

Service layer

business logic เช่น นักศึกษาคนนี้ลงวิชาที่ 6 ได้ไหมด้วยเกรดนี้ ถ้าไม่ได้ ให้แจ้งว่าต้องยื่นคำร้อง

Data access layer

แค่ insert, update, delete และ find ไม่มีกฎและไม่มี HTTP

- เป้าหมายคือขยายง่ายและใช้ซ้ำได้ เปลี่ยน framework เข้ามาแทนแค่ layer เดียวได้โดยไม่แตะ layer อื่น
- JSON กับ JavaScript object หน้าตาคล้ายกันแต่ไม่เหมือน ชื่อ field ของ JSON ต้องมีเครื่องหมายคำพูดคู่ ของ JavaScript ไม่ต้อง layer บนสุดเป็นคนแปลง

06

Layers in Pure Node.js

ยังไม่มี framework http **module** ทำส่วน HTTP และเราจัด layer เอง

project layout

```
src/
├── server.js        native HTTP server setup
├── router.js        route matching and HTTP handling
├── services/        business logic, no framework
└── repositories/    data access (file system or DB)
```

อาจารย์สร้าง layer ล่างสุดก่อน โดยใช้ array แทน database สี่ฟังก์ชันของ **repository** ก็คือ CRUD อีกครั้ง

repositories/subject-repository.js

```
const subjects = [];
export function findById(id) {
  return subjects.find((s) => s.id === id);
}
export function create(subject) {
  if (subjects.findIndex((s) => s.id === subject.id) >= 0) return false; // duplicate id
  subjects.push(subject);
  return subject;
}
export function update(id, subject) {
  const idx = subjects.findIndex((s) => s.id === id);
  if (idx < 0) return false; // fail fast
  subjects[idx] = { ...subject, id };
  return subjects[idx];
}
export function remove(id) {
  const idx = subjects.findIndex((s) => s.id === id);
  if (idx < 0) return false;
  subjects.splice(idx, 1);
  return true;
}
```

- ยังไม่มี key ที่สร้างอัตโนมัติ create จึงเช็ค id ซ้ำก่อน ถ้ามีแล้วให้ return false
- update และ remove ใช้ findIndex ถ้าได้เลขติดลบคือไม่เจอ แล้ว return false, item หรือ true ผู้เรียกจะรู้ว่าสำเร็จไหม
- เช็คกรณีล้มเหลวก่อนแล้ว return ทันที สไตล์นี้เรียกว่า fail fast

> **ทดสอบทีละ layer**
> ทดสอบ repository เดี่ยว ๆ ก่อนเขียน layer ถัดไป เรียกแต่ละฟังก์ชันจากไฟล์ทดสอบแล้วพิมพ์ผล ถ้าพังภายหลัง จะรู้ว่า layer ล่างใช้ได้แล้ว บั๊กจึงอยู่ใน code ใหม่

- ด้านบน **router** จับคู่แต่ละ method กับ URL ไปยังการเรียก **service** ทดสอบด้วย Postman: GET /subjects, GET /subjects/INT102, DELETE ตอบ 204 No Content และไม่มี body ส่วน POST กับ PUT ยัง "กำลังสร้าง" จนกว่าจะอ่าน body ได้ (ครั้งถัดไป)

07

อภิธานศัพท์

API
: Application Programming Interface: ช่องทางที่โปรแกรมเปิดให้โปรแกรมอื่นเรียกใช้

back end
: ส่วนบน server จัดการข้อมูล กฎ และ database แล้วตอบ front end ด้วย JSON

body
: ข้อมูลที่ request พกมาหลัง header สำหรับ API มักเป็น JSON

Boilerplate
: code ที่เกือบเหมือนกันทุกครั้ง เปลี่ยนแค่ชื่อ framework มีไว้ลดส่วนนี้

browser
: client ที่คนส่วนใหญ่ใช้ เช่น Chrome, Edge, Safari

business logic
: กฎของแอป เช่น การตรวจสอบ การคำนวณ และสิ่งที่ทำได้เมื่อไร

Cacheable
: เก็บข้อมูลที่ใช้บ่อยไว้ในหน่วยความจำที่เร็ว จะได้ไม่ต้องไปอ่าน database ที่ช้า hit คือมีอยู่แล้ว

client
: ฝั่งที่ร้องขอ มักเป็น browser แต่เป็นโปรแกรมหรืออุปกรณ์อะไรก็ได้

collection
: ชื่อ resource ที่ได้หลายรายการ เช่น /products คำตอบเป็น JSON array

CRUD
: Create, Read, Update, Delete: สี่อย่างพื้นฐานที่ทำกับข้อมูล

endpoint
: method กับ path หนึ่งคู่ที่ API ตอบ เช่น GET /api/subjects

entity

ข้อมูลใน database ตามที่เก็บ: แถวของ table พร้อมทุกคอลัมน์

framework

ชุดเครื่องมือที่ช่วยให้งานง่ายขึ้น แต่มีกรอบที่เราต้องเขียนตาม

front end

ส่วนที่ผู้ใช้เห็นและใช้ รันบน browser หรือโทรศัพท์

header

ข้อมูลเสริมของข้อความ ส่งเป็นบรรทัด name: value เช่น Content-Type

HTTP

HyperText Transfer Protocol: ภาษาข้อความธรรมดาที่ browser กับ web server ใช้คุยกัน ฝั่งหนึ่งส่ง request อีกฝั่งส่ง response

JSON

JavaScript Object Notation: รูปแบบข้อมูลที่เป็นข้อความ คนและโปรแกรมอ่านได้

kebab-case

คำต่อกันด้วยขีดกลาง เช่น order-items เป็นธรรมเนียมของ URI

layered system

แบ่ง code เป็น layer ตามหน้าที่ แต่ละ layer คุยกับ layer ข้างเคียงเท่านั้น

module

ไฟล์ code ที่โหลดเข้าไฟล์อื่นด้วย import

Postman

เครื่องมือส่ง HTTP method ใดก็ได้ไปที่ API ใช้ทดสอบก่อนที่ front end จะเสร็จ

repository

layer ที่อ่านเขียน database ไม่มี business rule และไม่มี HTTP

representation

รูปแบบที่ใช้ส่ง resource เช่น text, XML หรือ JSON ส่วนใหญ่ใช้ JSON

request

สิ่งที่ client ส่งมา: method, URL, header และบางครั้งมี body

resource

สิ่งที่ API ให้เข้าถึง เช่น product หรือ customer ในวิชานี้ตรงกับ table ใน database

response

สิ่งที่ server ส่งกลับ: status code, header และ body

REST

แนวทางออกแบบ web API: resource ตั้งชื่อด้วย URL และเลือก action ด้วย HTTP method

RESTful

API ที่ทำตามกฎของ REST ครบ ไม่ใช่แค่ใช้ HTTP กับ JSON

router

การจับคู่ HTTP method และ URL path กับฟังก์ชันที่จัดการ

server

โปรแกรม (หรือเครื่องที่รันมัน) ที่รอรับ request แล้วตอบกลับ

service

layer ที่เก็บ business rule เรียก repository และไม่รู้เรื่อง HTTP

SOAP

Simple Object Access Protocol: web service แบบเก่าที่ส่ง XML หนักกว่า แต่รองรับลายเซ็นดิจิทัล

soft delete

ทำเครื่องหมายว่าลบแล้วแทนการลบจริง ข้อมูลจะยังตามย้อนได้

Stateless

server ไม่จำอะไรระหว่าง request แต่ละ request ต้องพกข้อมูลที่ต้องใช้มาครบ

status code

ตัวเลขที่บอก client ว่าผลเป็นอย่างไร เช่น 200 OK, 201 Created, 404 Not Found, 500 Server Error

URI

Uniform Resource Identifier: ข้อความที่ระบุชื่อ resource เช่น /products/1 ส่วน URL คือ URI ที่บอกที่อยู่ด้วย

URL

ที่อยู่ของ resource ประกอบด้วย protocol, host, port และ path เช่น http://localhost:3000/user